

机器视觉-实践项目 3

国嘉通*

2025 年 9 月 26 日

摘要

机器视觉第三次实验课程主要包含两大部分的练习，一是频域平滑图像，二是频率锐化图像。对于频域平滑图像部分，分别从理想低通频域滤波、高斯低通频域滤波、巴特沃斯低通频域滤波三个方面进行详细阐释。对于频率锐化图像部分，分别从理想高通频域滤波、高斯高通频域滤波及巴特沃斯高通频率滤波。希望本篇文章是一篇内容翔实、方便随时翻阅复习学习 OpenCV 的学习笔记。

Now the CODE is available at:

<https://github.com/GplasT0810/Machine-Vision-Experiment.git>

长沙理工大学

卓越工程师学院 绿色智慧交通 2401 班

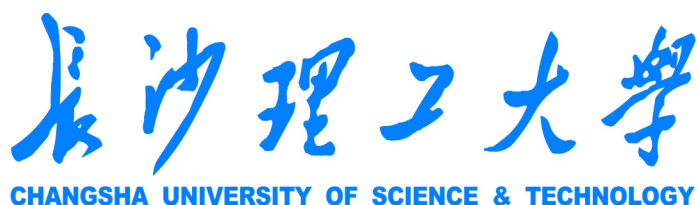
202404080608

国嘉通

*Tel.+86-166 8130 6085, E-mail:guojiatom2006@outlook.com

目录

1 频域平滑图像操作	3
1.1 理想低通频率滤波操作	3
1.2 高斯低通频率滤波操作	8
1.3 巴特沃斯低通频率滤波操作	16
2 频率锐化图像操作	19
2.1 理想高通频域滤波操作	19
2.2 高斯高通频域滤波	22
2.3 巴特沃斯高通频率滤波	25



Completed Time: Sep.22th.2025

Guo Jiatong , Green & Intelligent Transportation 2401
Elite Engineering School , Changsha University of Science & Technology.
Changsha 410114 , Hunan , China

1 频域平滑图像操作

本部分主要包含了理想低通频域滤波、高斯低通频域滤波以及巴特沃斯低通频域滤波三种低通滤波操作。

1.1 理想低通频率滤波操作

理想低通滤波（ILPF）的数学原理如下所示：

- 空域 $\rightarrow$ 频域（中心化）

先对一幅 $M \times N$ 灰度图像 $f(x, y)$ 做二维离散傅里叶变换（DFT）得到频谱 $F(u, v)$ 。

为把低频移到图像中心，应对 f 进行 $f \times (-1)^{(x+y)}$ 操作后，再调用 `cv2.dft`（或 `np.fft.fft2`）。

- 理想低通滤波器定义

给定截止频率 D_0 （非负实数），理想低通滤波器传递函数为：

$$H(u, v) = 1, \text{ if } D(u, v) \leq D_0$$

$$H(u, v) = 0, \text{ if } D(u, v) > D_0$$

其中 $D(u, v)$ 是频率点 (u, v) 到中心 $(\frac{M}{2}, \frac{N}{2})$ 的欧氏距离：

$$D(u, v) = \sqrt{(u - \frac{M}{2})^2 + (v - \frac{N}{2})^2}$$

- 频域滤波

滤波后频谱 $G(u, v) = H(u, v) \times F(u, v)$

相当于把高频分量直接删掉，只保留低频（图像的平滑区域、整体灰度分布）。

- 频域 $\rightarrow$ 空域（去中心化）

对 $G(u, v)$ 做逆 DFT（并取实部），再去中心化（乘以 $(-1)^{(x+y)}$ ），得到平滑后的空域图像 $g(x, y)$ 。

```
# 理想低通频域滤波
import cv2
import numpy as np
import matplotlib.pyplot as plt

def ideal_low_pass_filter(shape, cutoff_frequency):
    """
    创建理想低通滤波器
    :变量 1 shape: 图像形状 (rows, cols)
    :变量 2 cutoff_frequency: 截止频率
```

```

: return: 滤波器掩码
"""

rows, cols = shape
center_row, center_col = rows // 2, cols // 2

# 创建网格
u = np.arange(rows)
v = np.arange(cols)
u, v = np.meshgrid(u, v, indexing='ij')

# 计算距离中心的欧氏距离
distance = np.sqrt((u - center_row)**2 + (v - center_col)**2)

# 创建理想低通滤波器
filter_mask = np.zeros_like(distance)
filter_mask[distance <= cutoff_frequency] = 1

return filter_mask

def apply_ideal_low_pass_filter(image, cutoff_frequency):
    """
    应用理想低通滤波器
    :参数 1: image: 输入图像
    :参数 2: cutoff_frequency: 截止频率
    :return: 滤波后的图像
    """

    rows, cols = image.shape

    # 将图像转换为 float32
    img_float32 = np.float32(image)

    # 中心化
    mask = np.ones((rows, cols))
    mask[1::2, ::2] = -1
    mask[:, ::2, 1::2] = -1
    centered_img = img_float32 * mask

```

```

# 进行傅里叶变换
# 获取最佳 DFT 大小
optimal_rows = cv2.getOptimalDFTSize(rows)
optimal_cols = cv2.getOptimalDFTSize(cols)

# 创建扩展的复数图像
padded_img = np.zeros((optimal_rows, optimal_cols), dtype=np.float32)
padded_img[:rows, :cols] = centered_img

# 进行 DFT
dft = cv2.dft(padded_img, flags=cv2.DFT_COMPLEX_OUTPUT)

# 创建并应用理想低通滤波器
filter_mask = ideal_low_pass_filter((optimal_rows, optimal_cols),
↪cutoff_frequency)

# 将滤波器应用于 DFT 结果
dft_filtered = dft.copy()
dft_filtered[:, :, 0] *= filter_mask
dft_filtered[:, :, 1] *= filter_mask

# 进行逆傅里叶变换
idft = cv2.idft(dft_filtered)
filtered_img = idft[:, :, 0]

# 逆中心化
mask_padded = np.ones((optimal_rows, optimal_cols))
mask_padded[1::2, ::2] = -1
mask_padded[:, 1::2, 1::2] = -1

result_img = filtered_img * mask_padded

# 裁剪到原始大小
final_result = result_img[:rows, :cols]

```

```

# 归一化到 0-255 范围
final_result = cv2.normalize(final_result, None, 0, 255, cv2.NORM_MINMAX)
final_result = np.uint8(final_result)

return final_result

def main():
    image_path = '/Users/guo2006/myenv/Machine Vision Experiment/Machine Vision_
↳Experiment 3/xx.jpg'
    image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    if image is None:
        raise FileNotFoundError("图像文件未找到")

    # 定义不同的截止频率
    cutoff_frequencies = [30, 60, 120]

    # 创建图形显示结果
    plt.figure(figsize=(15, 10))

    # 对每个截止频率应用理想低通滤波
    for i, cutoff in enumerate(cutoff_frequencies):
        print(f"应用截止频率为 {cutoff} 的理想低通滤波器...")

        # 应用滤波器
        filtered_image = apply_ideal_low_pass_filter(image, cutoff)

        # 显示结果
        plt.subplot(2, 3, i + 1)
        plt.imshow(filtered_image, cmap='gray')
        plt.title(f'Cut-off Frequency D0 = {cutoff}')
        plt.axis('off')

    # 显示频域滤波器形状
    filter_shapes = []
    for cutoff in cutoff_frequencies:
        # 创建滤波器形状用于显示

```

```

    shape = image.shape
    optimal_rows = cv2.getOptimalDFTSize(shape[0])
    optimal_cols = cv2.getOptimalDFTSize(shape[1])
    filter_mask = ideal_low_pass_filter((optimal_rows, optimal_cols),
↪cutoff)
    filter_shapes.append(filter_mask)

# 显示滤波器形状
for i, (filter_mask, cutoff) in enumerate(zip(filter_shapes,
↪cutoff_frequencies)):
    plt.subplot(2, 3, i + 4)
    plt.imshow(filter_mask, cmap='gray')
    plt.title(f'Filter Shape (D0 = {cutoff})')
    plt.axis('off')

plt.tight_layout()
plt.show()

if __name__ == "__main__":
    main()

```

观察输出图像可知，当 $D_0 = 30$ 时，输出图像只保留了非常靠近中心的低频区域，低频滤波效果好，但图像严重模糊，边缘与细节几乎消失，出现明显的振铃效应。当 $D_0 = 60$ 时，图像相对来说保留了更多的中频，图像的模糊程度减轻，轮廓开始显现，但仍能看到不明显的轻微振铃效应。当 $D_0 = 120$ 时，图像的高频部分被保留的比例进一步增大，图像更接近原图，低频滤波效果更小，图像平滑效果变弱，振铃效应几乎不可见。

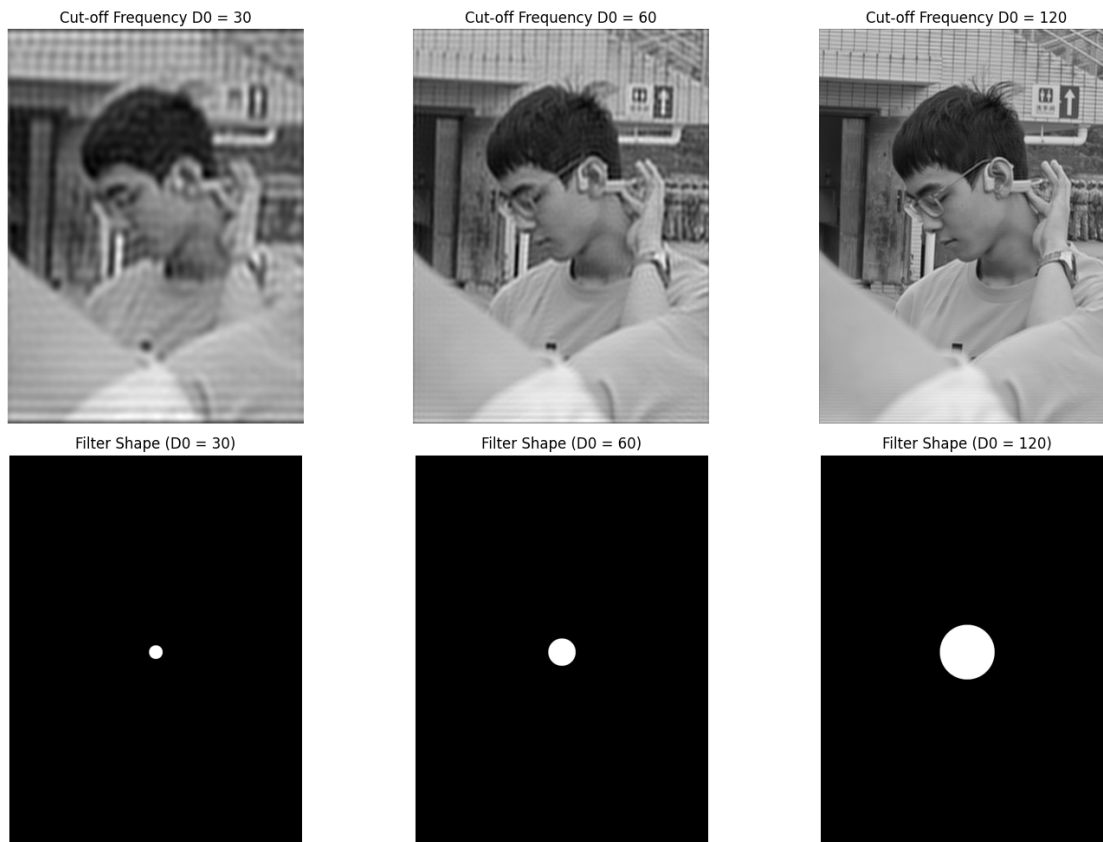
总归来说，核心步骤有四步：中心化 → DFT → 乘理想掩模 → 逆 DFT。

结论如下：截止频率 D_0 越大，保留的高频越多，图像越清晰； D_0 越小，平滑越强，振铃效应越明显。

应用截止频率为 30 的理想低通滤波器...

应用截止频率为 60 的理想低通滤波器...

应用截止频率为 120 的理想低通滤波器...



1.2 高斯低通频率滤波操作

相对于理想低通频率滤波以及以下将要提到的巴特沃斯低通频率滤波而言，高斯低通滤波只需要整体替换掉“掩模”部分即可直接跑出高斯平滑的图像处理结果。换言之，三种频域平滑操作的普通只有“掩模”上的操作不同。

高斯低通滤波的数学原理如下：

- 传递函数定义（频域）

与理想低通的“0/1 硬截断”不同，高斯低通采用“软截断”，即：

$$H(u, v) = e^{\frac{-D^2(u, v)}{2 \times D_0^2}}$$

其中， $D(u, v) = \sqrt{(u - \frac{M}{2})^2 + (v - \frac{N}{2})^2}$ 仍然是到频域中心的距离，对 D_0 有：

D_0 越大 → 指数衰减越慢 → 保留的高频越多 → 图像越清晰；

D_0 越小 → 高频被更快地压制 → 图像越模糊，但没有理想低通的振铃效应发生。

- 物理含义

频域乘上一个高斯面，相当于空域与原图做高斯卷积（相当于 `cv2.GaussianBlur` 函数）。

因此，GLPF 是“频域视角下的高斯模糊”，二者互为傅里叶变换对。


```

# 高斯低通频率滤波
import cv2
import numpy as np
import matplotlib.pyplot as plt

def gaussian_low_pass_filter(shape, cutoff_frequency):
    """
    创建高斯低通滤波器
    :参数 1 shape: 图像形状 (rows, cols)
    :参数 2 cutoff_frequency: 截止频率
    :return: 滤波器掩码
    """
    rows, cols = shape
    center_row, center_col = rows // 2, cols // 2

    # 创建网格
    u = np.arange(rows)
    v = np.arange(cols)
    u, v = np.meshgrid(u, v, indexing='ij')

    # 计算距离中心的欧氏距离
    distance = np.sqrt((u - center_row)**2 + (v - center_col)**2)

    # 创建高斯低通滤波器
    #  $H(u,v) = \exp(-D(u,v)^2 / (2 * D0^2))$ 
    filter_mask = np.exp(-(distance**2) / (2 * (cutoff_frequency**2)))

    return filter_mask

def apply_gaussian_low_pass_filter(image, cutoff_frequency):
    """
    应用高斯低通滤波器
    :参数 1 image: 输入图像
    :参数 2 cutoff_frequency: 截止频率
    :return: 滤波后的图像
    """

```

```

rows, cols = image.shape

# 将图像转换为 float32
img_float32 = np.float32(image)

# 中心化
mask = np.ones((rows, cols))
mask[1::2, ::2] = -1
mask[:, 1::2, 1::2] = -1
centered_img = img_float32 * mask

# 进行傅里叶变换
# 获取最佳 DFT 大小
optimal_rows = cv2.getOptimalDFTSize(rows)
optimal_cols = cv2.getOptimalDFTSize(cols)

# 创建扩展的复数图像
padded_img = np.zeros((optimal_rows, optimal_cols), dtype=np.float32)
padded_img[:rows, :cols] = centered_img

# 进行 DFT
dft = cv2.dft(padded_img, flags=cv2.DFT_COMPLEX_OUTPUT)

# 创建并应用高斯低通滤波器
filter_mask = gaussian_low_pass_filter((optimal_rows, optimal_cols), □
↪cutoff_frequency)

# 将滤波器应用于 DFT 结果
dft_filtered = dft.copy()
dft_filtered[:, :, 0] *= filter_mask
dft_filtered[:, :, 1] *= filter_mask

# 进行逆傅里叶变换
idft = cv2.idft(dft_filtered)
filtered_img = idft[:, :, 0]

```

```

# 逆中心化
mask_padded = np.ones((optimal_rows, optimal_cols))
mask_padded[1::2, ::2] = -1
mask_padded[:, 1::2] = -1

result_img = filtered_img * mask_padded

# 裁剪到原始大小
final_result = result_img[:rows, :cols]

# 归一化到 0-255 范围
final_result = cv2.normalize(final_result, None, 0, 255, cv2.NORM_MINMAX)
final_result = np.uint8(final_result)

return final_result

def display_frequency_spectrum(image, title=""):
    """
    显示图像的频谱
    """
    rows, cols = image.shape
    img_float32 = np.float32(image)

    # 获取最佳 DFT 大小
    optimal_rows = cv2.getOptimalDFTSize(rows)
    optimal_cols = cv2.getOptimalDFTSize(cols)

    # 创建扩展的复数图像
    padded_img = np.zeros((optimal_rows, optimal_cols), dtype=np.float32)
    padded_img[:rows, :cols] = img_float32

    # 进行 DFT
    dft = cv2.dft(padded_img, flags=cv2.DFT_COMPLEX_OUTPUT)

    # 将零频率分量移到中心
    dft_shift = np.fft.fftshift(dft)

```

```

# 计算幅度谱
magnitude_spectrum = 20 * np.log(cv2.magnitude(dft_shift[:, :, 0],
↪dft_shift[:, :, 1]) + 1)

return magnitude_spectrum

def main():
    print("高斯低通频域滤波")
    print("=" * 40)

    image_path = '/Users/guo2006/myenv/Machine Vision Experiment/Machine Vision_
↪Experiment 3/xx.jpg'
    image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    if image is None:
        raise FileNotFoundError("图像文件未找到，使用示例图像")

    print(f"图像尺寸: {image.shape}")

    cutoff_frequencies = [20, 50, 100]

    plt.figure(figsize=(12, 10))

    # 原图及其频谱图
    plt.subplot(4, 3, 1)
    plt.imshow(image, cmap='gray')
    plt.title('Original Image')
    plt.axis('off')

    spectrum_original = display_frequency_spectrum(image)
    plt.subplot(4, 3, 2)
    plt.imshow(spectrum_original, cmap='jet')
    plt.title('Original Spectrum')
    plt.axis('off')

    # 占位空图

```

```

plt.subplot(4, 3, 3)
plt.axis('off')

# 滤波图像 + 频谱 + 滤波器形状
for i, cutoff in enumerate(cutoff_frequencies):
    filtered_image = apply_gaussian_low_pass_filter(image, cutoff)
    spectrum_filtered = display_frequency_spectrum(filtered_image)
    filter_shape = gaussian_low_pass_filter(image.shape, cutoff)

    row = i + 1 # 从第 2 行开始

    # 滤波图像
    plt.subplot(4, 3, row * 3 + 1)
    plt.imshow(filtered_image, cmap='gray')
    plt.title(f'Filtered D0={cutoff}')
    plt.axis('off')

    # 滤波后频谱
    plt.subplot(4, 3, row * 3 + 2)
    plt.imshow(spectrum_filtered, cmap='jet')
    plt.title(f'Spectrum D0={cutoff}')
    plt.axis('off')

    # 滤波器形状
    plt.subplot(4, 3, row * 3 + 3)
    plt.imshow(filter_shape, cmap='gray')
    plt.title(f'Filter D0={cutoff}')
    plt.axis('off')

plt.tight_layout()
plt.show()

# 创建高斯滤波对比图
plt.figure(figsize=(12, 8))

plt.subplot(1, 4, 1)

```

```

plt.imshow(image, cmap='gray')
plt.title('Original')
plt.axis('off')

for i, cutoff in enumerate(cutoff_frequencies):
    filtered_image = apply_gaussian_low_pass_filter(image, cutoff)
    plt.subplot(1, 4, i + 2)
    plt.imshow(filtered_image, cmap='gray')
    plt.title(f'Gaussian Low Pass Filter D0 = {cutoff}')
    plt.axis('off')

plt.tight_layout()
plt.show()

if __name__ == "__main__":
    main()

```

观察图像可知，当 $D_0 = 20$ 时，高斯曲面衰减极快 $\rightarrow$ 最终图像只剩最低频，图像非常糊，但边缘干净，且无振铃效应。当 $D_0 = 50$ 时，对图像进行中等滤波，中频明显，图像细节被平滑，图像仍保持自然过渡。当 $D_0 = 100$ 时，图像的高频保留更多 $\rightarrow$ 处理后的图像接近原图，且模糊程度弱。

高斯低通滤波就是频域乘一个二维高斯函数 $e^{\frac{-D^2(u,v)}{2 \times D_0^2}}$ 。其相对于理想低通频率滤波操作的优势在于其没有明显振铃效应，因为高斯函数在空域也是有值的，不像理想低通频率滤波，具有“非黑即白”的掩膜。

高斯低通频域滤波

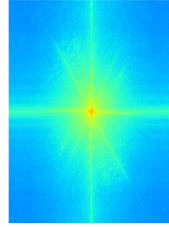
=====

图像尺寸：(1706, 1279)

Original Image



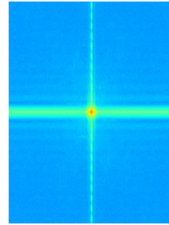
Original Spectrum



Filtered D0=20



Spectrum D0=20



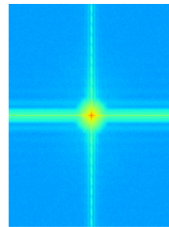
Filter D0=20



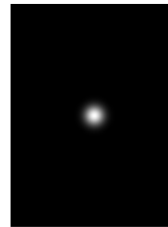
Filtered D0=50



Spectrum D0=50



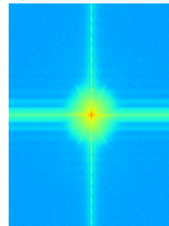
Filter D0=50



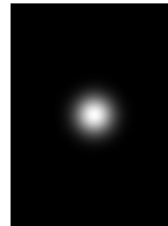
Filtered D0=100



Spectrum D0=100



Filter D0=100



Original



Gaussian Low Pass Filter D0 = 20



Gaussian Low Pass Filter D0 = 50



Gaussian Low Pass Filter D0 = 100



1.3 巴特沃斯低通频率滤波操作

巴特沃斯低通滤波的原理如下所示：

- 传递函数定义（频域）

巴特沃斯低通滤波的原理与理想低通滤波的“硬截”掩膜以及高斯低通滤波的“软截”掩膜都不同，巴特沃斯低通滤波使用 n 阶平滑曲线进行过渡：

$$H(u, v) = \frac{1}{1 + \left(\frac{D(u, v)}{D_0}\right)^{2n}}$$

其中， $D(u, v)$ 表示某一像素到频域中心距离（同上 1.1、1.2 小节）； D_0 表示截止频率（当 $D = D_0$ 时 $H = 0.5$ ，即 $-3dB$ 点）； n 表示阶数（order），取值需为正整数； n 越大 $\rightarrow$ 过渡带越陡 $\rightarrow$ 越接近理想低通，但振铃也越大。

当 $n=1$ 时，图像最平缓且基本无振铃效应，图像的模糊程度比高斯低通频率滤波略弱。

- 物理含义

频域乘上 BLPF 等价于空域与一个无旁瓣/弱旁瓣的“平滑核”做卷积，因此其能在“振铃”与“边缘保留”之间做折中，是实际工程里最常用的频域低通滤波操作。

```
# 巴特沃斯低通频率滤波
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 参数区
img_path = '/Users/guo2006/myenv/Machine Vision Experiment/Machine Vision_
↳Experiment 3/xx.jpg'
D0_list = [20, 40, 80] # 截止频率
orders = [2, 5] # 阶数

def pad_to_optimal(img):
    """ 扩展成 FFT 最佳尺寸 (2 的整数次幂) """
    h, w = img.shape
    opt_h = cv2.getOptimalDFTSize(h)
    opt_w = cv2.getOptimalDFTSize(w)
    padded = cv2.copyMakeBorder(img, 0, opt_h-h, 0, opt_w-w, borderType=cv2.
↳BORDER_CONSTANT, value=0)
    return padded, h, w
```



```

def butterworth_lowpass(shape, D0, n):
    """ 生成 n 阶 Butterworth 低通滤波器 """
    rows, cols = shape[:2]
    crow, ccol = rows//2, cols//2
    y, x = np.ogrid[:rows, :cols]
    dist_sq = (x - ccol)**2 + (y - crow)**2
    H = 1 / (1 + (dist_sq / (D0**2))**n)
    return H

def fft_filter(img, D0, n):
    """ 频域滤波并返回裁剪后的空域结果 """
    # 补零
    padded, orig_h, orig_w = pad_to_optimal(img)
    ph, pw = padded.shape
    H = butterworth_lowpass((ph, pw), D0, n)
    #FFT
    dft = cv2.dft(np.float32(padded), flags=cv2.DFT_COMPLEX_OUTPUT)
    dft_shift = np.fft.fftshift(dft)
    # 滤波
    filtered_dft = dft_shift * H[:, :, np.newaxis]
    #IFFT
    idft_shift = np.fft.ifftshift(filtered_dft)
    idft = cv2.idft(idft_shift)
    filtered = cv2.magnitude(idft[:, :, 0], idft[:, :, 1])
    # 裁剪回原尺寸 + 归一化
    filtered = filtered[:orig_h, :orig_w]
    cv2.normalize(filtered, filtered, 0, 255, cv2.NORM_MINMAX)
    return np.uint8(filtered)

# 读图
gray = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
assert gray is not None, "图片读取失败, 请检查路径"

fig, axes = plt.subplots(nrows=2, ncols=3, figsize=(9, 6),
    constrained_layout=True)
axes = axes.flatten() # 拉成一维

```

```

idx = 0
for i, n in enumerate(orders):
    for j, D0 in enumerate(D0_list):
        out = fft_filter(gray, D0, n)

        axes[idx].imshow(out, cmap='gray')
        axes[idx].set_title(f'order={n}, D0={D0}', fontsize=10)
        axes[idx].axis('off')
        idx += 1

plt.savefig('butterworth_results.jpg', dpi=300)
plt.show()

```



特性/滤波器	理想低通 (ILPF)	高斯低通 (GLPF)	巴特沃斯低通 (BLPF)
频域形状	两条 0/1 竖直线	高斯函数	平滑阶跃曲线
空域核	sinc 振荡	高斯	弱振荡/无负瓣
振铃效应	严重	无	可调 ($n \uparrow \rightarrow$ 振铃 $\uparrow$)
参数	D_0	D_0	$D_0 +$ 阶数 n
过渡带陡峭	最陡 (垂直)	最缓	中间 (n 控制)
视觉效果	边缘有涟漪	自然模糊	在“锐”与“糊”间折中
工程应用	教学/演示	快速平滑	频域增强/抗混叠首选

2 频率锐化图像操作

本部分主要包含了理想高通频域滤波、高斯高通频域滤波以及巴特沃斯高通频域滤波三种高通锐化操作。本质上说，只需要将上部分的低通掩膜改为高通掩膜，即可实现将对应的频域滤波图像操作转换为对应的频率锐化图像操作。

2.1 理想高通频域滤波操作

理想高通滤波的数学原理如下所示：

- 传递函数定义（频域）
其数学条件与理想低通完全互补：

$$H(u, v) = 0, \text{ if } D(u, v) \leq D_0$$

$$H(u, v) = 1, \text{ if } D(u, v) > D_0$$

即此操作将把图像中心低频全部砍断，只保留外围高频 $\rightarrow$ 图像的边缘、纹理、噪声被保留，平滑区域被抑制。

- 空域等价
频域 $\times$ IHPF = 空域与原图做“冲激核 - 低通 sinc 核”卷积，因此会出现明显振铃，且整体灰度被归零（需做灰度偏移或加回直流分量才能正常显示）。

```
# 理想高通频域滤波
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os
```

```

img_path = '/Users/guo2006/myenv/Machine Vision Experiment/Machine Vision_
↳Experiment 3/lisy.jpg'
DO_list = [20, 40, 80] # 截止频率
alpha = 1.0 # 锐化强度

def pad_to_optimal(img):
    h, w = img.shape
    opt_h = cv2.getOptimalDFTSize(h)
    opt_w = cv2.getOptimalDFTSize(w)
    padded = cv2.copyMakeBorder(img, 0, opt_h-h, 0, opt_w-w, borderType=cv2.
↳BORDER_CONSTANT, value=0)
    return padded, h, w

def ideal_highpass(shape, DO):
    """ 理想高通滤波器: 中心为 0, 其余为 1 """
    h, w = shape[:2]
    cy, cx = h//2, w//2
    y, x = np.ogrid[:h, :w]
    mask = ((x - cx)**2 + (y - cy)**2) >= DO**2 # 圆外 =1, 圆内 =0
    return mask.astype(np.float32)

def freq_filter(img, DO, filter_func):
    """ 通用频域滤波: filter_func 返回 H(u,v) """
    padded, orig_h, orig_w = pad_to_optimal(img)
    ph, pw = padded.shape
    H = filter_func((ph, pw), DO) # 生成滤波器
    dft = cv2.dft(np.float32(padded), flags=cv2.DFT_COMPLEX_OUTPUT)
    dft_shift = np.fft.fftshift(dft)
    filtered = dft_shift * (1 - H)[:,:,np.newaxis] # 高通 = 1 - 低通
    idft = cv2.idft(np.fft.ifftshift(filtered))
    out = cv2.magnitude(idft[:,:,:0], idft[:,:,:1])[:orig_h, :orig_w]
    cv2.normalize(out, out, 0, 255, cv2.NORM_MINMAX)
    return np.uint8(out)

def unsharp_mask(img, alpha=1.0):
    """ 空域锐化: 原图 + alpha*(原图 - 高斯模糊) """

```

```

    blur = cv2.GaussianBlur(img, (9, 9), 2)
    sharp = cv2.addWeighted(img, 1+alpha, blur, -alpha, 0)
    return sharp

gray = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
assert gray is not None, "图片读取失败"

fig, ax = plt.subplots(2, 3, figsize=(10, 6), constrained_layout=True)
ax = ax.flatten()

for idx, D0 in enumerate(D0_list):
    # 理想高通滤波
    hp = freq_filter(gray, D0, ideal_highpass)

    # 锐化
    sharp = unsharp_mask(hp, alpha=alpha)

    # 显示
    ax[idx].imshow(hp, cmap='gray')
    ax[idx].set_title(f'Ideal HP D0={D0}')
    ax[idx].axis('off')

    ax[idx+3].imshow(sharp, cmap='gray')
    ax[idx+3].set_title(f'HP+Sharp D0={D0}')
    ax[idx+3].axis('off')

plt.show()

```

从实验得到的结果图中不难看出，当 $D_0=20$ 时，图像只保留了最高频 → 边缘极锐，但出现了明显的振铃效应（同心波纹）。当 $D_0 = 40$ 时，图像的中高频保留 → 图像边缘变粗，振铃效应减弱。当 $D_0 = 80$ 时，图像的更多低频漏进来 → 边缘开始发“灰”，接近原图。

Ideal HP D0=20



HP+Sharp D0=20



Ideal HP D0=40



HP+Sharp D0=40



Ideal HP D0=80



HP+Sharp D0=80



2.2 高斯高通频域滤波

高斯高通滤波的数学原理如下所示：

- 传递函数定义（频域）

高斯高通频域滤波与高斯低通频域滤波互为“1-”关系：

$$H(u, v) = 1 - e^{-\frac{D^2(u, v)}{2 \times D_0^2}}$$

D_0 越大 → 指数衰减越慢 → 保留的低频越多 → 边缘越弱；

D_0 越小 → 高频相对增强 → 边缘越锐，但始终无振铃效应（高斯核无旁瓣）。

- 空域等价

频域 × GHPF = 空域做“原图 - 高斯模糊图”，也就是经典的 Unsharp Mask (USM) 的频域版本。

```
# 高斯高通频域滤波
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os
```

```

img_path = '/Users/guo2006/myenv/Machine Vision Experiment/Machine Vision_
↳Experiment 3/lisy.jpg'
D0_list = [20, 40, 80] # 截止频率
alpha = 1.0 # 锐化强度

def pad_to_optimal(img):
    h, w = img.shape
    opt_h = cv2.getOptimalDFTSize(h)
    opt_w = cv2.getOptimalDFTSize(w)
    padded = cv2.copyMakeBorder(img, 0, opt_h-h, 0, opt_w-w, borderType=cv2.
↳BORDER_CONSTANT, value=0)
    return padded, h, w

def gauss_highpass(shape, D0):
    """ 高斯高通滤波器  $H(u,v) = 1 - \exp(-D^2/2D0^2)$  """
    h, w = shape[:2]
    cy, cx = h//2, w//2
    y, x = np.ogrid[:h, :w]
    dist2 = (x - cx)**2 + (y - cy)**2
    H = 1 - np.exp(-dist2 / (2 * D0**2))
    return H.astype(np.float32)

def freq_filter(img, D0, filter_func):
    """ 通用频域滤波 """
    padded, orig_h, orig_w = pad_to_optimal(img)
    ph, pw = padded.shape
    H = filter_func((ph, pw), D0)
    dft = cv2.dft(np.float32(padded), flags=cv2.DFT_COMPLEX_OUTPUT)
    dft_shift = np.fft.fftshift(dft)
    filtered = dft_shift * H[:, :, np.newaxis]
    idft = cv2.idft(np.fft.ifftshift(filtered))
    out = cv2.magnitude(idft[:, :, 0], idft[:, :, 1])[:orig_h, :orig_w]
    cv2.normalize(out, out, 0, 255, cv2.NORM_MINMAX)
    return np.uint8(out)

```

```

def unsharp_mask(img, alpha=1.0):
    blur = cv2.GaussianBlur(img, (9, 9), 2)
    sharp = cv2.addWeighted(img, 1+alpha, blur, -alpha, 0)
    return sharp

gray = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
assert gray is not None, "图片读取失败"

fig, axes = plt.subplots(2, 3, figsize=(10, 6), constrained_layout=True)
axes = axes.flatten()

for idx, D0 in enumerate(D0_list):
    # 高斯高通
    hp = freq_filter(gray, D0, gauss_highpass)

    # 锐化
    sharp = unsharp_mask(hp, alpha=alpha)

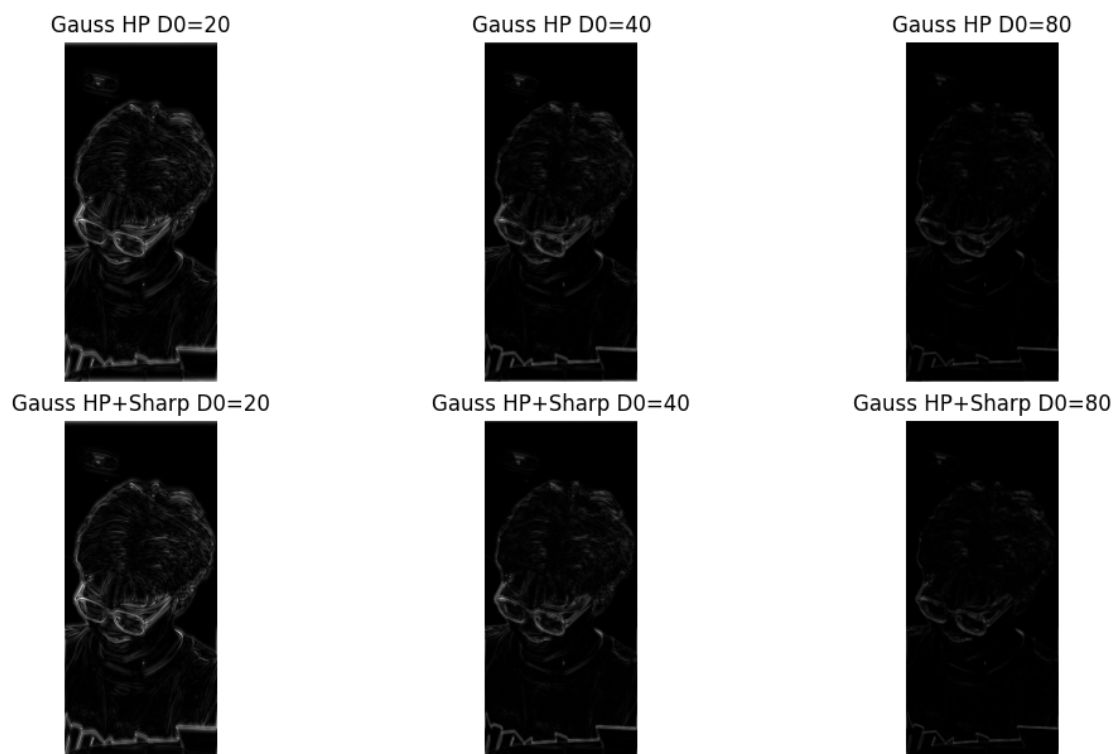
    # 显示
    axes[idx].imshow(hp, cmap='gray')
    axes[idx].set_title(f'Gauss HP D0={D0}')
    axes[idx].axis('off')

    axes[idx+3].imshow(sharp, cmap='gray')
    axes[idx+3].set_title(f'Gauss HP+Sharp D0={D0}')
    axes[idx+3].axis('off')

plt.show()

```

观察实验效果图可以得知：当 $D_0 = 20$ 时，图像的高斯衰减极快 → 几乎只剩最高频 → 图像边缘锐利，多噪点，无振铃效应。当 $D_0 = 40$ 时，图像的中高频增强 → 图像边缘相对自然，对比度提升，仍无振铃效应。当 $D_0 = 80$ 时，低频漏得更多 → 边缘变柔和。



2.3 巴特沃斯高通频率滤波

巴特沃斯高通滤波的原理如下所示：

- 传递函数定义（频域）
与巴特沃斯低通互为“1-”关系：

$$H(u, v) = \frac{1}{1 + (\frac{D_0}{D(u, v)})^{2n}}$$

$$= 1 - \frac{1}{1 + (\frac{D(u, v)}{D_0})^{2n}}$$

其中，n 为阶数（为 >0 的整数），当 $n \uparrow \rightarrow$ 过渡带越陡 $\rightarrow$ 越接近理想高通滤波，振铃效应略增；

D_0 为截止频率（当 $D = D_0$ 时 $H = 0.5$ ，即 $-3dB$ 点）。

- 空域含义
频域 $\times$ BHPF = 空域做“原图 - 巴特沃斯低通图”，因此能在“边缘锐度”与“振铃控制”之间折中，是实际高频增强/细节提取的常用核。

```
# 巴特沃斯高通频率滤波
import cv2
import numpy as np
```

```

import matplotlib.pyplot as plt
import os

img_path = '/Users/guo2006/myenv/Machine Vision Experiment/Machine Vision_
↳Experiment 3/lisy.jpg'
D0_list = [20, 40, 80] # 截止频率
orders = [2, 5] # 阶数
alpha = 1.0 # 锐化强度

def pad_to_optimal(img):
    h, w = img.shape
    opt_h = cv2.getOptimalDFTSize(h)
    opt_w = cv2.getOptimalDFTSize(w)
    padded = cv2.copyMakeBorder(img, 0, opt_h-h, 0, opt_w-w, borderType=cv2.
↳BORDER_CONSTANT, value=0)
    return padded, h, w

def butterworth_highpass(shape, D0, n):
    """ 巴特沃斯高通:  $H(u,v) = 1 / (1 + (D0/D)^{2n})$  """
    h, w = shape[:2]
    cy, cx = h//2, w//2
    y, x = np.ogrid[:h, :w]
    dist = np.sqrt((x - cx)**2 + (y - cy)**2)
    dist[dist == 0] = 1e-8 # 防除 0
    H = 1 / (1 + (D0 / dist)**(2 * n))
    return H.astype(np.float32)

def freq_filter(img, D0, n, filter_func):
    padded, orig_h, orig_w = pad_to_optimal(img)
    ph, pw = padded.shape
    H = filter_func((ph, pw), D0, n)
    dft = cv2.dft(np.float32(padded), flags=cv2.DFT_COMPLEX_OUTPUT)
    dft_shift = np.fft.fftshift(dft)
    filtered = dft_shift * H[:, :, np.newaxis]
    idft = cv2.idft(np.fft.ifftshift(filtered))
    out = cv2.magnitude(idft[:, :, 0], idft[:, :, 1])[:orig_h, :orig_w]

```

```

cv2.normalize(out, out, 0, 255, cv2.NORM_MINMAX)
return np.uint8(out)

def unsharp_mask(img, alpha=1.0):
    blur = cv2.GaussianBlur(img, (9, 9), 2)
    sharp = cv2.addWeighted(img, 1+alpha, blur, -alpha, 0)
    return sharp

gray = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
assert gray is not None, "图片读取失败"

fig, axes = plt.subplots(2, 6, figsize=(15, 5), constrained_layout=True)

for row, n in enumerate(orders):
    for col, D0 in enumerate(D0_list):
        # 巴特沃斯高通
        hp = freq_filter(gray, D0, n, butterworth_highpass)

        # 锐化
        sharp = unsharp_mask(hp, alpha=alpha)

        # 显示
        axes[row, col].imshow(hp, cmap='gray')
        axes[row, col].set_title(f'Order={n} D0={D0}')
        axes[row, col].axis('off')

        axes[row, col+3].imshow(sharp, cmap='gray')
        axes[row, col+3].set_title(f'Order={n} D0={D0} +Sharp')
        axes[row, col+3].axis('off')

plt.show()

```

通过观察图像不难发现，当 $D_0 = 20$ 时，过渡带相对窄 $\rightarrow$ 边缘锐利，噪点被放大，几乎无振铃效应 ($n=2$)。当 $D_0 = 40$ 时，中高频增强 $\rightarrow$ 边缘自然，对比度提升，细节丰富。当 $D_0 = 80$ 时，低频漏得更多 $\rightarrow$ 边缘柔和。且弱把阶数 n 调到 4 以上，边缘会再锐一点，但振铃仍远小于理想高通。

Order=2 D0=20



Order=2 D0=40



Order=2 D0=80



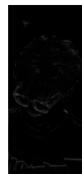
Order=2 D0=20 +Sharp



Order=2 D0=40 +Sharp



Order=2 D0=80 +Sharp



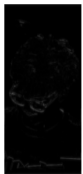
Order=5 D0=20



Order=5 D0=40



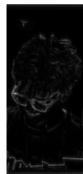
Order=5 D0=80



Order=5 D0=20 +Sharp



Order=5 D0=40 +Sharp



Order=5 D0=80 +Sharp

